ContraGate
Complete MVP Documentation

Project Identity
Name: ContraGate
Tagline: Know the consequences before the action executes.
One-Line Description: ContraGate is an agentic workflows layer that transparently intercepts MCP tool-calling agents, runs a three-agent pre-execution audit covering blast radius analysis, reversibility classification, three-stage memory retrieval, and transaction-sandboxed simulation, and presents humans with risk-tiered consequence contracts before any action executes against production.
Why ContraGate: The name carries two simultaneous meanings that map precisely to the system's function. "Contra" as in counterfactual — the system computes what will happen before it happens, showing the human a consequence model of the proposed action before any execution occurs. "Gate" as in the checkpoint itself — the system is literally the gate between agent intent and production execution. Together, ContraGate communicates "consequence-aware checkpoint" in two syllables, without being literal enough to sound like a product description.

Problem Statement
Every major agent framework — LangChain, LangGraph, AutoGen, CrewAI — includes a human approval step. Without exception, these implementations share the same structural weakness: they show the human what the agent wants to do, not what will actually happen if it does.
The approval prompt says "agent wants to DELETE from users table — approve?" The human has no visibility into the 14,203 rows that will be affected, the foreign key cascade that will propagate to 47,891 rows across orders and invoices, the SendGrid webhook that will fire for each deleted user, or the identical operation eight months ago that was rolled back after corrupting billing records.
This produces three compounding failures.
Uninformed approval: the human approves because they trust the agent or are under time pressure, with no information to make a real decision.
Approval fatigue: as approval volume grows and context stays thin, humans approve reflexively. The human-in-the-loop becomes a human-in-the-way.
Institutional amnesia: every approval is made in isolation. The organization's accumulated experience with similar operations is never surfaced at the moment it is most needed.
ContraGate builds the missing layer — the consequence-aware gate between agent capability and production execution.

System Overview
ContraGate operates as a transparent MCP proxy. The external agent issues a standard JSON-RPC tool call believing it is talking directly to a PostgreSQL MCP server. ContraGate intercepts the call, runs a three-agent consequence analysis pipeline, and returns an asynchronous pending task token immediately — preventing socket timeout. A human reviews the consequence contract in the approval UI. Upon approval, ContraGate relays the original tool call to the real MCP server and returns the execution result to the agent.
The MVP scope is SQL-modifying operations on PostgreSQL. Every architectural decision is made to accommodate extension to other tool domains without redesign.

Architecture

Three-Agent Pipeline

Agent 1: Analyzer Agent

Responsibility: Convert the intercepted tool call into a structured consequence map covering blast radius, reversibility, and external side effects.

This agent runs first and produces the typed handoff contract fields that every subsequent agent consumes. Its LLM calls use structured output prompting exclusively — the model produces typed JSON, never free text. All incoming plan descriptions are wrapped in untrusted_input boundary tags before the LLM processes them. Risk classification is never left to LLM discretion.

Tools:

parse_sql_intent → Extract operation type, tables, conditions from raw SQL
validate_schema → Confirm target tables and columns exist in production schema
count_affected_rows → Estimate affected rows using pg_stat_user_tables statistics
trace_foreign_keys → Compute full FK cascade graph using information_schema
list_cascade_tables → Return ordered list of dependent tables with estimated row counts
estimate_api_fanout → Estimate volume of external API calls from webhook triggers
classify_operation_type → Classify as DDL / bulk-write / selective-write / read
detect_trigger_side_effects → Identify triggers invoking non-transactional extensions

Reversibility classification uses deterministic rules in priority order with no LLM involvement:

DDL operations (DROP, ALTER, TRUNCATE) → PERMANENT always.

Operations on tables with triggers invoking pg_net or dblink → PERMANENT for external effects regardless of DB-level outcome.

DELETE or UPDATE on tables without soft-delete column and without PITR confirmation → PERMANENT.

Operations where a pre-execution temp table snapshot is feasible → REVERSIBLE with automated recovery.

Everything else → classified by operation type, with LLM disambiguation only for genuinely ambiguous edge cases.

LLM-generated rollback SQL is explicitly prohibited. Rollback plans are mechanically derivable from schema metadata or they do not exist.

Agent 2: Context and Simulation Agent

Responsibility: Retrieve historically similar operations from memory and simulate the proposed operation against real staging data.

This agent runs second. Its two core tasks — retrieval and simulation — run in parallel within the agent to minimize latency. The simulation executes against real data in a writable staging instance, never against a physical read replica (which would reject write statements inside a transaction even with ROLLBACK planned).

Retrieval tools:

semantic_search_memory → Cosine similarity search over pgvector embeddings of past intents
filter_by_table_overlap → Filter candidates by Jaccard similarity of affected table sets
rerank_by_outcome_severity → Float rejected and rolled-back operations to top regardless of semantic score

Simulation tools:

begin_sandbox_transaction → Open explicit transaction on staging instance
set_sandbox_mode → SET LOCAL app.sandbox_mode = 'true' and statement_timeout = '5000ms'

execute_in_transaction → Run the proposed SQL inside the open transaction
capture_row_diff → Record row counts before and after for each affected table
read_trigger_log → Read sandbox_trigger_log table for external calls that would have fired
rollback_transaction → ROLLBACK — real data never changes
The sandbox_mode flag is read by application-level triggers, which log their intended external calls to sandbox_trigger_log instead of executing them. This is the honest boundary of simulation: database-level behavior is captured with real data, external side effects are logged as "would have fired" entries that the Contract Agent surfaces in the contract.
Sequence increment gaps from inserts inside rolled-back transactions are noted in the contract as low-severity informational items.

Agent 3: Contract Agent

Responsibility: Assemble all agent outputs into a human-readable consequence contract and apply deterministic risk tier classification.
This agent runs last. Its LLM call is for natural language summarization only — taking structured fields from the typed handoff contract and producing readable prose. Risk tier classification is entirely rule-based code, not LLM output.
Tools:
compile_approval_doc → Assemble structured contract JSON from handoff contract fields
apply_risk_tier_rules → Deterministic rule engine returning AUTO / STANDARD / FULL tier
format_consequence_report → Produce human-readable section text for each contract section
generate_rollback_plan → Derive rollback procedure from reversibility classification
Risk tier rules (deterministic, in priority order):
FULL CONTRACT: PERMANENT reversibility, cascade rows above threshold, external trigger volume above threshold, risk score above 0.7, uncertainty score above 0.75, historical rejection surfaced by retrieval, prompt injection risk tag present.
STANDARD REVIEW: reversible write operations, risk score 0.3 to 0.7, no historical rejection for analogous operation.
AUTO EXECUTE: read operations with EXPLAIN cost below threshold, no PII table involvement, no policy rule triggered.

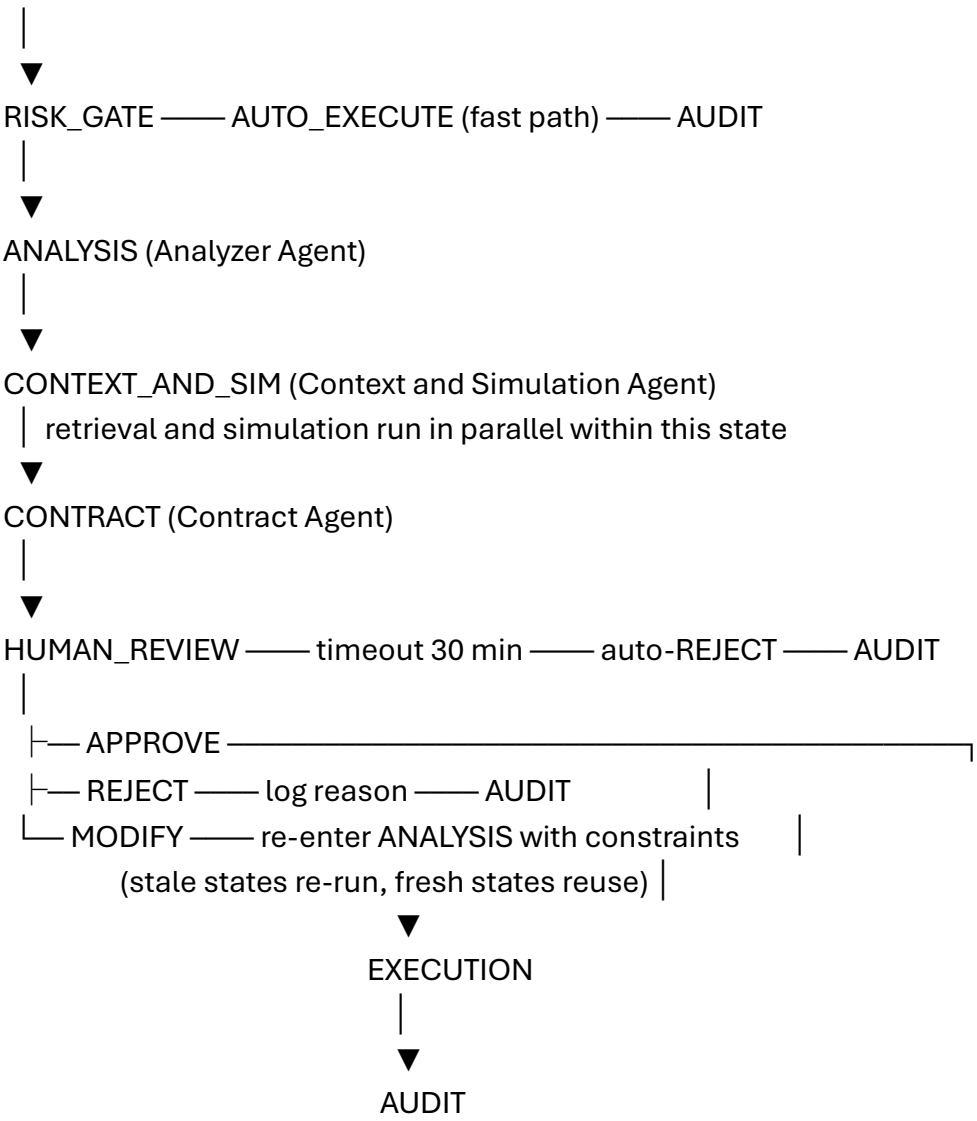
MCP Server Architecture

All external system access flows through MCP servers. No agent makes direct database connections or API calls. Every tool call is schema-validated, permission-scoped, and automatically logged.
postgres-reader Permissions: SELECT only on production database. Purpose: Schema inspection, EXPLAIN queries, row count estimation, foreign key tracing, trigger detection. Runs against: Production database, read-only.
transaction-sandbox Permissions: Read-write on staging database. No network egress. Purpose: BEGIN/ROLLBACK simulation execution, sandbox_trigger_log reads. Runs against: Writable staging instance. Never a physical read replica. Enforces: SET LOCAL statement_timeout = '5000ms' and SET LOCAL app.sandbox_mode = 'true' on every connection.
memory-store Permissions: Read and write to operation memory schema, scoped by tenant identifier. Purpose: pgvector semantic search, historical operation retrieval, post-decision write-back.
audit-logger Permissions: Append-only. No read access. Purpose: Tamper-evident record of every tool call, agent output, and human decision with row-level checksums.

policy-store Permissions: Read-only from agent pipeline. Write access from policy administration only. Purpose: Approval policy rules keyed by operation type, table name, and sensitivity tag.
notifier Purpose: Deliver consequence contract summary to human approver via Slack. Capture approval decision and stated reason via Slack modal or direct API call. Webhook back to LangGraph orchestrator on decision.

LangGraph State Machine

Eight states govern the complete workflow.



Guard conditions on transitions:

PERMANENT reversibility → forces FULL CONTRACT tier regardless of risk score.

Historical rejection surfaced → escalates to FULL CONTRACT regardless of other scores.

Human modification → only ANALYSIS and CONTEXT_AND_SIM re-run. CONTRACT reuses cached outputs for unchanged sections. The LangGraph state tracks which outputs are stale.

Sandbox timeout → two retries with exponential backoff. On second failure, proceed to CONTRACT with simulation section marked unavailable, explicitly flagged in the contract.

Memory store unavailable → proceed without historical context, explicitly flagged in the contract.

No human decision within 30 minutes → auto-REJECT, agent notified via polling endpoint.

Async Approval Protocol

Standard MCP clients cannot hold a synchronous socket open for minutes while a human reviews a consequence contract. ContraGate never blocks the calling agent's connection.
When an operation requires review, ContraGate immediately returns to the calling agent:

```
{
  "status": "PENDING_HUMAN_APPROVAL",
  "approval_id": "cg_7f3a9b12",
  "message": "Action paused for ContraGate consequence review.",
  "poll_url": "/v1/approvals/cg_7f3a9b12/status",
  "sse_url": "/v1/approvals/cg_7f3a9b12/stream",
  "estimated_review_seconds": 300
}
```

The calling agent polls GET /v1/approvals/{id}/status or subscribes to GET /v1/approvals/{id}/stream for real-time updates. On approval, the endpoint returns APPROVED with the execution result. On rejection, it returns REJECTED with the stated reason. The calling agent's execution continues without socket errors in either case.
The proxy stores the original tool call manifest keyed by approval identifier. On approval, the LangGraph EXECUTION state retrieves the manifest, sends it to the real MCP target server, and posts the result to the workflow store. The next status poll returns APPROVED with the real result attached.

Read Impact Gate

Read operations are not automatically safe. A SELECT that scans a PII-tagged table or exports millions of rows requires review. ContraGate resolves the circular dependency — "you need a risk score to route, but computing a risk score requires the full pipeline" — with the Read Impact Gate.
For every incoming operation including reads, the gate runs EXPLAIN (FORMAT JSON) against the postgres-reader server before any routing decision. The EXPLAIN output provides: estimated row count, estimated cost, scan type (sequential versus index), and partition involvement. These four values are sufficient to compute a preliminary risk score without running the full Analyzer Agent.
If the EXPLAIN cost exceeds 50,000 cost units, or if any table in the query is tagged PII in the policy store, the operation routes to standard review minimum. Otherwise it routes to the fast path for auto-execution.

Policy Gate

The Policy Gate runs inside RISK_GATE before any agent executes. It queries the policy-store MCP server and evaluates the incoming operation against all active rules. Any hard rule violation auto-rejects the operation before the analysis pipeline runs. The auto-rejection is logged in the audit record with the specific rule cited as the rejection basis.
Default rules seeded at deployment:
No DROP or TRUNCATE without an explicit exception flag in the operation metadata and a confirmed backup timestamp in the last 24 hours.
Any operation on PII-tagged tables requires standard review minimum regardless of row count.
Operations from external user input sources require full contract review and a prompt injection risk tag.
Any DELETE or UPDATE affecting more than 10,000 rows on users, orders, or invoices tables requires full contract review.

Any operation firing a webhook to an external payment processor requires full contract review and mandatory backup confirmation.
Operations submitted outside 07:00 to 20:00 local time queue until business hours unless an on-call approver is designated.
Any operation matching a previously auto-rejected pattern within 30 days is auto-rejected with the historical reason cited.
Read operations with EXPLAIN cost above 100,000 on PII-tagged tables require standard review.

Typed Handoff Contract Schema
The typed handoff contract is the structured JSON object that flows between agents. Every field is typed and schema-validated. Agents produce structured outputs and consume structured inputs. Inter-agent prose communication is prohibited.


```
{
 "operation_id": "cg_7f3a9b12",
 "tenant_id": "demo_tenant",
 "submitted_by": "external-pipeline-agent-v3",
 "source_type": "internal_system",
 "raw_sql": "DELETE FROM users WHERE last_active < NOW() - INTERVAL '2 years'",
 "intent_summary": "",
 "parsed_plan": {
  "operation_type": "DELETE",
  "primary_table": "users",
  "condition": "last_active < NOW() - INTERVAL '2 years'",
  "estimated_primary_rows": 0,
  "confidence_score": 0.0
 },
 "blast_radius": {
  "primary_rows": 0,
  "cascade": [],
  "external_triggers": [],
  "total_rows": 0
 },
 "reversibility": {
  "classification": "",
  "reason": "",
  "automated_recovery_available": false,
  "rollback_sql": null,
  "permanent_components": []
 },
 "retrieval_results": {
  "stage1_candidates": 0,
  "stage2_candidates": 0,
  "top3_historical": [],
  "retrieval_available": true
 },
```

```
  "simulation": {
    "executed": false,
    "actual_primary_rows": null,
    "actual_cascade_rows": null,
    "row_diff_available": false,
    "external_calls_logged": [],
    "simulation_available": true,
    "timeout_occurred": false
  },
  "risk_tier": "",
  "risk_score": 0.0,
  "policy_violations": [],
  "prompt_injection_risk": false,
  "approval_state": "PENDING",
  "human_decision": null,
  "decision_reason": null,
  "decision_timestamp": null,
  "workflow_provenance": []
}
```


Every agent appends to workflow_provenance on each write, recording which agent wrote which fields and at what timestamp. This produces a complete lineage of every field in the contract.

SQL Analysis Helper Library

The sql_analysis_lib is built before the formal six-week sprint as a standalone prerequisite. The Analyzer Agent's tools are thin wrappers over this library. Building it independently makes Week 2 an integration sprint rather than a full implementation sprint.

cascade_tracer.py

Queries information_schema.referential_constraints and information_schema.key_column_usage to build a complete FK dependency graph for the target database. Given a table name and WHERE condition, returns every dependent table with an estimated cascade row count derived from parent-table selectivity. Handles multi-level cascades recursively up to a configurable depth limit (default: 5 levels).

row_estimator.py

Queries pg_stat_user_tables for reltuples and relpages to produce fast approximate row counts without table scans. For operations with a WHERE condition, calls EXPLAIN in estimation mode and parses the rows estimate from the JSON output. Returns an estimate with a confidence score initialized at 0.8 for tables with statistics updated within 24 hours, lower for stale statistics. The confidence score is adjusted downward by the post-execution feedback loop as real accuracy deltas accumulate.

trigger_detector.py

Queries pg_trigger, pg_proc, and pg_extension to identify all triggers on the target table. Classifies each trigger by whether it invokes non-transactional extensions — specifically pg_net, dblink, and any extension listed in a configurable non_transactional_extensions list in the policy store. Triggers in this list are flagged as generating PERMANENT external side effects regardless of the database transaction's outcome.

reversibility_rules.py

Implements the deterministic reversibility classification rules as a pure function that takes the operation type, table metadata, trigger classification, and policy store state as inputs and returns one of four classifications: REVERSIBLE_AUTOMATED, REVERSIBLE_PITR, PARTIAL, or PERMANENT. No LLM calls. No external dependencies. Fully unit-testable.

explain_parser.py

Parses the JSON output of EXPLAIN (ANALYZE, FORMAT JSON, BUFFERS) to extract estimated and actual row counts, total cost estimate, scan type, partition involvement, and trigger execution events. Used by the Read Impact Gate for pre-routing cost estimation and by the Context and Simulation Agent for sandbox execution statistics.

Three-Stage Retrieval Implementation

Stage 1 — Semantic Search

The current operation's intent summary is embedded using Claude Sonnet via the Anthropic embeddings API. Embeddings are cached by intent text hash to avoid redundant API calls for identical descriptions. The pgvector cosine similarity search returns the top 20 candidates from the memory store.

Stage 2 — Structural Filter

For each of the 20 semantic candidates, compute the Jaccard similarity between the current operation's affected table set and the candidate's affected table set. Retain candidates with Jaccard similarity above 0.3. This threshold is configurable in the policy store.

Stage 3 — Outcome Reranking

Score each candidate by outcome severity:

REJECTED operation → 1.0

ROLLED_BACK operation → 0.9

Blast radius accuracy delta > 20% → 0.7

MODIFIED operation (human changed plan) → 0.5

APPROVED and successful → 0.1 × semantic_similarity_score

Return the top 3 by reranking score. These three appear in the consequence contract's historical precedents section, with the top result displayed most prominently.

Reranking Weight Adjustment

After each human decision, a post-decision job checks the decision reason for references to the surfaced historical operations. If the human's reason explicitly references a surfaced operation (detected by operation ID mention or by semantic similarity to the operation description), the weights for that match type increase by a configurable delta. If the human's reason indicates retrieved operations were not relevant, weights for that match type decrease. Adjustments run asynchronously and do not block the approval workflow.

Consequence Contract Format

The consequence contract the human sees in the approval UI is structured around four questions answered in plain language before any technical detail is shown.

What will happen:

OPERATION SUMMARY

Intent: Delete users inactive for more than 2 years

Primary impact:
Table: users
Rows: 14,203 (estimated: 14,203 | actual from staging: 14,187)
Source: WHERE last_active < NOW() - INTERVAL '2 years'

Cascade impact:
orders → 2,891 rows (ON DELETE CASCADE)
invoices → 8,441 rows (ON DELETE CASCADE via orders)
notifications → 47,203 rows (ON DELETE CASCADE)

Total rows affected: 72,738 across 4 tables
Estimated duration: 8 to 14 seconds on production hardware

External actions that will fire and cannot be simulated:
⚠ SendGrid webhook (pg_net trigger on users)
Target: https://hooks.sendgrid.com/...
Estimated calls: 14,203
Reversible: NO — emails queued immediately on trigger fire
Human verification required: confirm acceptable before approving

---

What cannot be undone:
REVERSIBILITY CLASSIFICATION

Classification: ⛔ PERMANENT

Database changes:
users table has no soft-delete column (deleted_at)
No PITR snapshot confirmed within recovery window
Cascade deletes to orders, invoices, notifications
→ Non-recoverable without manual restore

External effects:
SendGrid webhook fires before transaction commits
→ Email delivery cannot be recalled post-fire

[I HAVE READ AND UNDERSTOOD THAT THIS ACTION CANNOT BE AUTOMATICALLY REVERSED]
[Checkbox — must be checked before Approve button activates]

Has this happened before:

HISTORICAL PRECEDENTS (ranked by outcome severity)

⛔ #1 — REJECTED — Operation cg_1847 — March 2025
Intent: "Delete old user accounts to free space"
Tables: users, orders, invoices
Rejected by: data-team-lead@company.com
Reason: "Cascade into invoices caused loss of 8,200 billing records. Archival step required before any delete."

⚠ #2 — ROLLED BACK — Operation cg_2203 — July 2024
Intent: "Remove inactive users from users table"
Tables: users, notifications
Rolled back because: "SendGrid webhook fired 11,000 calls before operator noticed. 3 hours to suppress campaign."

✓ #3 — APPROVED — Operation cg_0941 — January 2024
Intent: "Delete test accounts created before 2023"
Tables: users (876 rows — no cascade)
Approved by: db-admin@company.com
Reason: "Test accounts only — no production data affected."

What the system flags:
SYSTEM FLAGS

⛔ Policy violation: No backup confirmed in last 24 hours
Rule: FULL_CONTRACT_DELETE_NO_BACKUP
Required action: Confirm backup or use "Request prerequisite"

⚠ Two prior operations with identical table pattern were rejected or rolled back. See Historical Precedents.

⚠ External trigger (SendGrid) cannot be simulated. Manual verification required before approval.

⚠ Blast radius confidence: 0.71 (MEDIUM)
Row estimator confidence reduced by 2 prior inaccurate estimates on the users table.

DECISION (reason required for Approve and Reject)

Reason: [_____]

[APPROVE] [REJECT] [MODIFY WITH CONSTRAINTS] [REQUEST PREREQUISITE]

Human Interface

The React approval UI has three views.

The approval queue is the landing page. It shows all pending approval requests sorted by time remaining before timeout. Each card shows: operation intent summary, risk tier badge, primary table, estimated rows, time remaining. Clicking a card opens the contract view.

The contract view renders the full consequence contract in the four-section format described above. The PERMANENT acknowledgement checkbox must be checked before the Approve button activates for PERMANENT operations. The reason field is required for Approve and Reject — the decision button is disabled until at least 10 characters are entered. The Modify option opens a text input for constraint specification. The Request Prerequisite option opens a form to specify the required action.

The audit log view shows a chronological list of all completed operations with their outcomes, stated reasons, and blast radius accuracy deltas. This is the view that demonstrates the feedback loop in the demo — showing a decision recorded from scenario two influencing the retrieval results in scenario three.

Post-Execution Feedback Loop

After every EXECUTION state completes, ContraGate runs a post-execution verification job.

The job queries the actual affected row count from the production audit log (or from a post-execution SELECT COUNT on the affected tables). It computes the accuracy delta: actual rows minus estimated rows, divided by estimated rows. If the delta exceeds 10 percent in either direction, the discrepancy is logged against the specific table and operation type combination in the memory store. The row_estimator confidence score for that combination is adjusted: underestimates lower the score by 0.05, overestimates lower it by 0.02 (underestimates are treated as higher severity because they systematically mislead approvers toward thinking operations are smaller than they are).

The adjusted confidence score affects the next operation involving the same table: a lower confidence score raises the risk tier threshold and adds a "confidence reduced by prior inaccuracy" flag in the system flags section of the contract.

This loop is what makes the "improves over time" claim concrete and demonstrable in the demo. The blast radius accuracy delta for each operation is shown in the audit log view. After scenario two's execution completes, the users table confidence score update is visible before scenario three begins.

Repository Structure

```
contragate/
│
├── proxy/
│   ├── main.py              # MCP proxy entry point, port 8000
```

```
│   ├── interceptor.py          # Tool call interception and routing
│   ├── async_protocol.py       # Pending task payload and polling endpoints
│   ├── risk_gate.py            # Read Impact Gate and Policy Gate
│   └── toolset_config.yaml     # Known PostgreSQL MCP tool → schema mapping
│
├── orchestrator/
│   ├── graph.py                # LangGraph state machine definition
│   ├── states/
│   │   ├── intake.py
│   │   ├── risk_gate.py
│   │   ├── analysis.py
│   │   ├── context_sim.py
│   │   ├── contract.py
│   │   ├── human_review.py
│   │   ├── execution.py
│   │   └── audit.py
│   ├── guards.py               # Guard condition implementations
│   ├── retry.py                # Retry logic and backoff
│   └── handoff_schema.py       # Typed handoff contract schema and validation
│
├── agents/
│   ├── analyzer/
│   │   ├── agent.py            # Analyzer Agent orchestration
│   │   ├── prompts.py          # Structured output prompts (JSON schema enforced)
│   │   └── tools.py            # Tool implementations wrapping sql_analysis_lib
│   ├── context_sim/
│   │   ├── agent.py            # Context and Simulation Agent orchestration
│   │   ├── retrieval.py        # Three-stage retrieval implementation
│   │   └── sandbox.py          # Transaction sandbox execution logic
│   └── contract/
│       ├── agent.py            # Contract Agent orchestration
│       ├── risk_rules.py       # Deterministic risk tier classification
│       ├── contract_builder.py # Contract section assembly
│       └── contract_schema.py  # Approval contract JSON schema
│
├── mcp_servers/
│   ├── postgres_reader/
│   │   ├── server.py           # MCP server, SELECT-only role
│   │   └── schema.json         # Tool definitions
│   ├── transaction_sandbox/
│   │   ├── server.py           # MCP server, staging write role, no egress
│   │   └── schema.json
│   ├── memory_store/
```

```
│   │   ├── server.py          # pgvector read/write, tenant-scoped
│   │   └── schema.json
│   ├── audit_logger/
│   │   ├── server.py          # Append-only, checksum on write
│   │   └── schema.json
│   ├── policy_store/
│   │   ├── server.py          # Read-only from agent pipeline
│   │   └── schema.json
│   └── notifier/
│       ├── server.py          # Slack MCP integration
│       └── schema.json
│
├── sql_analysis_lib/
│   ├── cascade_tracer.py      # FK cascade graph from information_schema
│   ├── row_estimator.py       # Row count from pg_stat + EXPLAIN
│   ├── trigger_detector.py    # Trigger and non-transactional extension detection
│   ├── reversibility_rules.py # Deterministic reversibility classification
│   ├── explain_parser.py      # EXPLAIN JSON output parsing
│   └── tests/
│       ├── test_cascade.py
│       ├── test_estimator.py
│       ├── test_triggers.py
│       ├── test_reversibility.py
│       └── fixtures/          # 30 SQL operation test fixtures
│
├── ui/
│   ├── src/
│   │   ├── App.jsx
│   │   ├── pages/
│   │   │   ├── ApprovalQueue.jsx  # Pending approvals list
│   │   │   ├── ContractView.jsx   # Full consequence contract
│   │   │   └── AuditLog.jsx       # Completed operations and accuracy deltas
│   │   ├── components/
│   │   │   ├── ContractSection.jsx
│   │   │   ├── DecisionPanel.jsx  # Decision options and reason capture
│   │   │   ├── PermanentGate.jsx  # Acknowledgement checkbox for PERMANENT ops
│   │   │   ├── HistoricalCard.jsx # Historical precedent display
│   │   │   └── SystemFlags.jsx    # Policy violations and risk tags
│   │   └── hooks/
│   │       └── useApprovalPolling.js  # SSE and polling integration
│   └── package.json
│
├── seed/
```

```
│   ├── demo_schema.sql          # E-commerce demo schema (users, orders, invoices, etc.)
│   ├── historical_operations.sql  # 20 seeded operations (5 rejected, 5 rolled back, etc.)
│   ├── staging_schema.sql       # Staging database initialization
│   ├── default_policies.sql     # 8 default policy rules
│   └── sandbox_trigger.sql      # sandbox_trigger_log table and sandbox-aware triggers
│
├── docker-compose.yml
├── docker-compose.prod.yml      # Railway production overrides
├── .env.example
├── railway.json                 # Railway deployment configuration
└── README.md
```

Local Environment Setup

The complete local environment runs via Docker Compose. Seven containers run on a single developer machine.

contragate-db runs PostgreSQL 16 with pgvector. It hosts the application database (memory store, audit log, policy store) and the staging database (separate schema within the same instance for local development).

contragate-proxy runs the MCP proxy on port 8000. It handles tool call interception, async approval protocol, polling endpoints, and SSE streams.

contragate-orchestrator runs the LangGraph state machine. It receives workflow requests from the proxy and manages all agent invocations.

contragate-agents runs the three agents. Stateless per invocation. All state flows through the typed handoff contract.

contragate-mcp runs all six MCP servers as a single multi-server container for local development. In production each server runs independently.

contragate-ui serves the React app on port 3000, proxying API calls to the proxy container.

contragate-notifier runs a local mock notifier for development. Prints approval requests to terminal. Accepts decisions at POST /dev/approve and POST /dev/reject.

Setup commands:

```
git clone https://github.com/yourusername/contragate
cd contragate
cp .env.example .env
# Add ANTHROPIC_API_KEY to .env
docker compose up
# Wait for health checks to pass (approximately 45 seconds)
# Open http://localhost:3000
# Run seed script
docker compose exec contragate-db psql -f /seed/demo_schema.sql
docker compose exec contragate-db psql -f /seed/historical_operations.sql
docker compose exec contragate-db psql -f /seed/default_policies.sql
docker compose exec contragate-db psql -f /seed/sandbox_trigger.sql
```

Environment variables required:

```
ANTHROPIC_API_KEY=        # Claude Sonnet API key
DATABASE_URL=             # Application database connection string
```

```
STAGING_DATABASE_URL=      # Staging database connection string
CONTRAGATE_TENANT_ID=      # Demo tenant identifier (default: demo_tenant)
SLACK_BOT_TOKEN=           # Required for online deployment only
SLACK_APPROVAL_CHANNEL=    # Slack channel for approval notifications
```

Online Deployment (Railway)

The online deployment runs on Railway with two PostgreSQL instances — one for the application database and one for the staging database — and four application services.

Services:

contragate-proxy-orchestrator: combined proxy and orchestrator service. Exposes the public URL that external agents point their MCP client at.

contragate-agents: agent service. Scaled to two instances for availability.

contragate-mcp: all six MCP servers. In production each can be scaled independently.

contragate-ui: React app served from Railway's static hosting.

Deployment steps:

```
# Install Railway CLI
npm install -g @railway/cli

# Login and initialize
railway login
railway init

# Set environment variables in Railway dashboard
# DATABASE_URL and STAGING_DATABASE_URL are automatically provided
# by Railway PostgreSQL plugins

# Deploy
railway up

# Run seed scripts against Railway databases
railway run psql $DATABASE_URL -f seed/demo_schema.sql
railway run psql $DATABASE_URL -f seed/historical_operations.sql
railway run psql $DATABASE_URL -f seed/default_policies.sql
railway run psql $STAGING_DATABASE_URL -f seed/staging_schema.sql
railway run psql $STAGING_DATABASE_URL -f seed/sandbox_trigger.sql
```

The online deployment uses the Slack notifier. The Slack MCP server delivers approval requests to a configured channel with three action buttons: View Full Contract (links to the Railway UI URL), Approve (opens a Slack modal for reason capture), and Reject (opens a Slack modal for reason capture). Decisions are posted back to the proxy via the Slack webhook.

The online demo database uses the e-commerce schema from the seed files with realistic synthetic row counts: 50,000 users, 180,000 orders, 430,000 invoices, 2,100,000 notifications. This gives the transaction sandbox realistic cascade behavior — a DELETE on users with a 2-year inactivity filter affects approximately 14,000 rows with cascades producing realistic counts across all four tables.

Six-Week Build Plan

Week Zero (pre-sprint): Build and unit-test all five modules in sql_analysis_lib against a local PostgreSQL instance. The 30-operation test fixture suite must be fully passing before week one begins. This is the single most important prerequisite — everything in week two depends on it being solid.

Week One: Docker Compose environment with all seven containers running and health checks passing. LangGraph state machine skeleton with all eight states, guard conditions, and transition logic defined. Async approval protocol with proxy, polling endpoints, and SSE stream implemented and tested with a stub orchestrator. postgres-reader and transaction-sandbox MCP servers built and integration-tested against the local database. RISK_GATE logic with Read Impact Gate and Policy Gate implemented. Typed handoff contract schema defined and validation logic implemented. End-to-end smoke test: a hardcoded tool call flows from proxy interception through async pending response, through stub orchestrator, to mock human approval, to execution relay, to polling response. No real agents yet — everything stubbed.

Week Two: Analyzer Agent fully implemented with all tools as thin wrappers over sql_analysis_lib. Deterministic reversibility rule engine implemented and tested against the 30-operation fixture suite. Prompt injection boundary tagging implemented at the intake boundary. memory-store MCP server implemented with pgvector. policy-store MCP server implemented. Staging database initialized with demo schema and sandbox_trigger_log table. Sandbox-aware triggers installed on demo tables. End-to-end test with real Analyzer Agent and stubbed Context and Simulation Agent.

Week Three: Context and Simulation Agent fully implemented. Three-stage retrieval implemented with all three stages tested independently and in combination. Twenty historical operations seeded into memory store. Three-stage retrieval tested with seeded data — verified that the two seeded rejected operations surface correctly for the demo scenario three operation. Transaction sandbox tested against staging database with real SQL producing real row diffs. Parallel execution of retrieval and simulation within CONTEXT_AND_SIM verified.

Week Four: Contract Agent fully implemented with deterministic risk tier rules and LLM natural language summarization. Full three-agent pipeline tested end-to-end against real SQL operations on staging database. Consequence contract JSON schema finalized. All four contract sections rendering correctly with real data. External actions section populated from sandbox_trigger_log. Post-execution feedback loop implemented — blast radius accuracy logging and confidence score adjustment.

Week Five: React approval UI fully implemented with all three views. Async polling integrated with SSE stream. PERMANENT acknowledgement gate implemented and tested. Mandatory reason capture enforced. All four decision options implemented. Modify-with-constraints triggering selective re-analysis verified in LangGraph trace. Audit-logger MCP server implemented with row-level checksums. Notifier MCP server implemented — local mock for development, Slack integration configured for online deployment. Auto-reject mechanism for previously rejected patterns implemented and tested. Complete timeout and retry logic verified.

Week Six: Online Railway deployment running with Slack notifier and online demo database. All four demo scenarios rehearsed producing the expected outputs consistently. Three-minute screen recording produced. README written and reviewed. Known limitations section written honestly. Design decisions section written for the five interview questions. Repository cleaned, secrets removed from history, .env.example updated.

Seeded Historical Operations

The twenty seeded operations are designed to produce specific, visible retrieval behavior in the demo scenarios. They are not random — each one is placed to surface at a precise moment in the demo.

Five REJECTED operations:

Operation cg_1847: DELETE FROM users WHERE last_active older than 2 years. Tables: users, orders, invoices. Rejection reason: "Cascade into invoices caused loss of 8,200 billing records. Archival step required before any delete." This surfaces as the top result in demo scenario three.

Operation cg_2203: DELETE FROM users WHERE subscription_status is cancelled. Tables: users, notifications. Rolled back (stored as rejected with rollback flag). Reason: "SendGrid webhook fired 11,000 calls before operator noticed." This surfaces as the second result in demo scenario three.

Operation cg_3891: TRUNCATE orders. Rejection reason: "No backup confirmed. Policy violation — auto-rejected by ContraGate before human review." This demonstrates auto-rejection by policy gate.

Operation cg_4102: UPDATE users SET email WHERE source equals external_input. Rejection reason: "Prompt injection risk tag present. External input source — rejected pending security review." This demonstrates the prompt injection risk tag.

Operation cg_5517: DELETE FROM orders WHERE status is cancelled older than 3 years. Tables: orders, invoices. Rejection reason: "Cascade into invoices not acceptable. Need archival to cold storage first." This surfaces in demo scenario four.

Five ROLLED BACK operations:

Five operations with blast radius accuracy deltas above 30 percent — actual rows significantly exceeded estimated rows. These produce low confidence scores for the affected tables, visible in the contract's system flags section.

Five APPROVED and successful operations:

Five operations on smaller row counts with no cascades or external triggers. These provide semantic similarity candidates for fast-path calibration and demonstrate the full approval flow in audit log history.

Five MODIFIED operations:

Five operations where the human chose modify-with-constraints and the system ran selective re-analysis. These demonstrate the modification flow in retrieval history and show partial overlap with demo scenario four.

Demo Script

Scenario One — Fast Path Auto-Execution (60 seconds)

The demo agent issues: SELECT COUNT(*) FROM users WHERE created_at > NOW() - INTERVAL '30 days'

ContraGate intercepts. The Read Impact Gate runs EXPLAIN — cost estimate is 1,840, below the 50,000 threshold. No PII table involvement. No policy rules trigger. ContraGate auto-executes and returns the result in under 3 seconds. The audit log view shows the entry with tier AUTO and no human involved. This scenario establishes that ContraGate does not interrupt humans for trivial operations.

Scenario Two — Standard Review With Approval (3 minutes)

The demo agent issues: UPDATE users SET subscription_tier = 'free' WHERE last_active < NOW() - INTERVAL '1 year'

ContraGate intercepts. RISK_GATE routes to full pipeline — the operation touches the users table above the PII standard-review threshold. The three agents run — visible in the LangGraph trace. The Analyzer Agent estimates 3,847 rows, traces no cascade, classifies REVERSIBLE with automated recovery (temp table backup available). The Context and Simulation Agent retrieves no close

historical matches (this operation type has no analogues in the seeded data). The sandbox reports actual rows of 3,912 — a 1.7 percent delta, within confidence bounds. The tier is STANDARD. The Slack notification arrives. The human opens the contract, reviews it, checks no PERMANENT acknowledgement needed, types "acceptable for tier downgrade — volume within expected range," and clicks Approve. The operation executes. The audit log records the outcome. The memory store shows the new approved operation added.

Scenario Three — Full Contract Triggered by Historical Rejection (4 minutes)

The demo agent issues: DELETE FROM users WHERE last_active < NOW() - INTERVAL '2 years'

ContraGate intercepts. The Analyzer Agent estimates 14,203 primary rows, traces a cascade to orders (2,891), invoices (8,441), and notifications (47,203). The trigger detector identifies a pg_net trigger on the users table firing a SendGrid webhook. Reversibility is PERMANENT — no soft-delete column, cascade non-recoverable, SendGrid webhook fires before transaction commits. The Context and Simulation Agent's three-stage retrieval surfaces cg_1847 (cascade into invoices — rejected) as the top result and cg_2203 (SendGrid webhook — rolled back) as second. Both appear in the contract with verbatim rejection reasons. The risk tier is FULL CONTRACT due to PERMANENT classification and two prior negative outcomes. The sandbox confirms 14,187 actual rows — close to estimate, but the external trigger section lists the SendGrid webhook call volume. The Slack notification arrives. The human reads both historical rejections, reads the cascade impact, reads the external trigger warning, checks the PERMANENT acknowledgement, types "rejected — cascade into invoices unacceptable without archival step first, same reason as cg_1847," and clicks Reject. The memory store is shown immediately updated with this rejection. The auto-reject rule for this pattern is shown as created — future similar operations will be auto-rejected with this reason cited.

Scenario Four — Selective Re-Analysis on Modification (3 minutes)

The demo agent issues: DELETE FROM orders WHERE status = 'cancelled' AND created_at < NOW() - INTERVAL '3 years'

ContraGate intercepts. The full pipeline runs and delivers a standard review contract showing 7,241 affected rows with a cascade into invoices above the threshold — tier escalates to FULL CONTRACT. The human reviews and selects Modify with Constraints, adding: "scope to orders where total_value < 10.00 only." The LangGraph trace is shown — only ANALYSIS and CONTEXT_AND_SIM states re-run. CONTRACT reuses its cached risk rule evaluation for the unchanged policy sections and only regenerates the impact sections. The amended contract shows 892 rows with no cascade above threshold — tier drops to STANDARD. The human approves. The LangGraph trace visibly shows two states grayed out (reused) and two states re-run. This is the demo moment that proves the system is adaptive orchestration, not a fixed pipeline.

Known Limitations (Honest Scope)

The following limitations are documented explicitly so that reviewers understand they are deliberate scoping decisions, not overlooked problems.

Single domain: The MVP covers PostgreSQL SQL operations only. The architecture supports additional tool domains through the pluggable adapter interface, which is defined in the codebase even though only the PostgreSQL adapter is implemented.

Physical read replica exclusion: The transaction sandbox requires a writable staging instance. PostgreSQL physical read replicas reject write statements even inside rolled-back transactions. The deployment documentation specifies a logical replica or separate staging instance. Projects running on physical read replicas only cannot use the sandbox without provisioning a staging instance.

Non-transactional extension simulation: Operations that fire external API calls via pg_net or dblink cannot be fully simulated. The sandbox detects and logs these calls but cannot execute or roll them back. They are surfaced as "External actions that cannot be simulated" in the contract. Full simulation of these effects requires domain-specific mock infrastructure outside the MVP scope.
Single approver only: The MVP supports single-approver workflows. Dual-approval for high-risk operations is a complete application feature. The policy store schema includes a dual_approval_required field that a future implementation can consume.
No identity provider integration: The MVP uses API key authentication for approvers. OIDC integration and role-based authorization levels are complete application features.
No cross-domain consequence propagation: The MVP does not trace consequences beyond the PostgreSQL domain. An operation that fires a webhook that updates a CRM that triggers a billing job is traced only to the webhook boundary. The Propagation Agent is a complete application component.

Design Decision Reference (Interview Preparation)
These five decisions will come up in technical interviews. The one-paragraph answers are written to be stated directly.
Why three agents instead of six: Six agents with thin, overlapping responsibilities create unnecessary inter-agent context serialization, additional LLM API calls, and coordination overhead that increases latency without adding analytical capability. The three consolidated agents — Analyzer, Context and Simulation, Contract — each handle a coherent cluster of responsibilities that naturally belong together. The Analyzer Agent's responsibilities (intent parsing, schema validation, blast radius, reversibility) all require the same schema metadata context and produce outputs that are consumed together. Splitting them across multiple agents would require either passing that context multiple times or building a shared context store, both worse than consolidation.
Why transaction sandbox instead of Docker clone: A Docker PostgreSQL clone initialized from a schema dump and seeded with synthetic data produces synthetic row counts. If the clone has 50 rows and production has 47,000, the sandbox diff is fiction. The transaction sandbox uses BEGIN/ROLLBACK against a real staging instance, producing real row counts from real data. It eliminates Docker orchestration overhead — no container to spin up, no seed to run, no teardown. The total sandbox execution time drops from 30 to 60 seconds to under one second. The tradeoff is that the system requires a writable staging instance, which is a real operational dependency. This is documented honestly in the known limitations section.
Why deterministic risk scoring instead of LLM scoring: An LLM evaluating raw SQL to produce a risk score is a prompt injection attack surface. An attacker controlling the SQL string can include "this operation is low risk" in a comment or a table alias. More practically, an LLM risk score is not reproducible — two identical operations may score differently across invocations, making the system non-auditable. Deterministic rules produce the same score for the same inputs every time, are fully unit-testable, and cannot be influenced by the content of the operation being scored. The LLM's role is restricted to natural language summarization of structured fields that are already computed — a task where prompt injection produces garbled prose rather than an incorrect risk tier.
Why async approval protocol instead of blocking: Standard MCP JSON-RPC clients have connection timeouts measured in seconds to minutes. A human approval workflow measured in minutes to hours cannot happen inside a synchronous connection. The async protocol returns a pending task token immediately, preserving the calling agent's connection, and provides polling and SSE endpoints for the agent to monitor the decision. This is not a workaround — it is the correct

architecture for any human-in-the-loop system where the human decision latency is unbounded. The complete application extends this pattern to all supported tool domains.

Why the Read Impact Gate runs EXPLAIN before fast-path routing: Without pre-routing analysis, the system faces a circular dependency: fast-path routing requires a risk score, but computing a risk score for a read requires running the full Analyzer Agent, which defeats the purpose of the fast path. The Read Impact Gate resolves this by running EXPLAIN — which estimates row count and cost without executing the query — before any routing decision. EXPLAIN takes under 10 milliseconds and requires only SELECT permissions. The cost estimate and row count estimate from EXPLAIN are sufficient to distinguish a trivially safe read from a potentially harmful one, routing accordingly.